

Minibatch Adaptive Optimization via Forward Automatic Differentiation for Designing Efficient TSK Fuzzy Systems

Hui Zhang, Wei Peng, *Member, IEEE*, Chengdong Li, *Member, IEEE*, Bo Sun, *Member, IEEE*, and Qinglai Wei, *Senior Member, IEEE*

Abstract—Takagi–Sugeno–Kang (TSK) fuzzy systems have gained considerable attention in machine learning applications, such as regression and classification, owing to their strong non-linear processing capability and good interpretability. However, mainstream optimization algorithms often suffer from high computational costs (including time complexity and memory overhead) when designing TSK fuzzy systems. To address this issue, this study proposes a novel forward automatic differentiation-based minibatch adaptive optimization (MBAO–FAD) algorithm for efficient training of TSK fuzzy systems. First, this study employs FAD in TSK fuzzy systems to enable practical gradient descent, thereby entirely eliminating the demand for backward propagation. Second, a matrix-free, Jacobian-vector product-based gradient estimation method is proposed to enable rapid gradient evaluation for all fuzzy parameters in only a single forward propagation pass. Furthermore, a minibatch fuzzy parameter update strategy based on non-negative matrix factorization is proposed; this strategy considerably reduces computational memory requirements. Finally, extensive experiments are conducted to demonstrate the training stability and computational competitiveness of the proposed MBAO–FAD algorithm across various regression tasks. Compared with other benchmark algorithms, MBAO–FAD reduces computational time by an average of over 40% while maintaining accuracy and low memory consumption.

Index Terms—Fuzzy system, automatic differentiation, machine learning, matrix factorization, parameter optimization.

I. INTRODUCTION

FUZZY logic systems [1], particularly the Takagi–Sugeno–Kang (TSK) fuzzy systems [2], exhibit remarkable universal approximation capability and interpretability [3]. These systems have been widely applied in machine learning tasks [4]–[9], especially in solving typical regression problems [10]–[12]. To design TSK fuzzy systems, numerous effective

training algorithms have been proposed, which can be broadly categorized into the following types.

Least Squares Estimation (LSE). This algorithm is commonly employed to optimize the consequent parameters of TSK fuzzy systems, specifically the linear parameters in the rule output stage, for minimizing the discrepancy between predicted and actual values. Previously, as a classical optimization algorithm, LSE has been extensively used in designing improved TSK fuzzy systems, such as adaptive neuro–fuzzy inference systems [13] and functionally weighted single-input-rule-module-based fuzzy inference systems [14]. Furthermore, researchers have proposed enhanced LSE variants to improve the robustness and adaptability of TSK fuzzy systems, with regularized LSE [15] and weighted LSE [16] being the most representative.

Evolutionary Algorithm (EA). In this algorithm, antecedent membership functions and consequent parameters are encoded as individuals within a population. Genetic operators are then iteratively applied to generate subsequent generations until a globally optimal solution is obtained [17]. Zhao *et al.* [18] proposed a multiobjective optimization method for designing self-generating interpretable hybrid fuzzy logic systems. Vikas *et al.* [19] developed a fuzzy adaptive dynamic strength pareto EA with Borda Ranking (FDSPEA2–BR) to address multiobjective optimization problems involving trade-offs between power cost and user comfort.

Gradient Descent (GD). Typically, TSK fuzzy systems require derivative evaluation when solving machine learning tasks. Most GD algorithms rely on computing gradients of the loss function with respect to antecedent and consequent parameters and subsequently optimizing these parameters along the negative gradient direction to locate global and/or local minima. Backward propagation remains one of the most prevalent strategies for gradient computation [20] and is crucial for enhancing the approximation capability of TSK fuzzy systems in machine learning applications [21], [22]. Furthermore, this algorithm is frequently employed in fuzzy neural networks [23].

The aforementioned algorithms have demonstrated remarkable success in machine learning tasks employing TSK fuzzy systems. However, according to Wu [24] and Cui *et al.* [25], these algorithms still present several practical limitations. When handling large-scale and/or high-dimensional data, all current algorithms suffer from prohibitive computational costs and substantial memory requirements.

Received November 18, 2025.

This work was supported by the National Natural Science Foundation of China under Grant 62573272, Grant 62173216, and Grant 62076150. (Corresponding authors: Wei Peng; Chengdong Li.)

Hui Zhang and Bo Sun are with the School of Control Science and Engineering, Shandong University, Jinan 250014, China (e-mail: 202234949@mail.sdu.edu.cn, sunbo@sdu.edu.cn).

Wei Peng and Chengdong Li are with the School of Information and Electrical Engineering, Shandong Jianzhu University, Jinan 250101, China (e-mail: pengwei19@sdjzu.edu.cn, lichengdong@sdjzu.edu.cn).

Qinglai Wei is with the State Key Laboratory for Management and Control of Complex Systems, Institute of Automation, Chinese Academy of Sciences, Beijing 100190, China, also with the School of Artificial Intelligence, University of Chinese Academy of Sciences, Beijing 100049, China, and also with the Institute of Systems Engineering, Macau University of Science and Technology, Macau 999078, China (e-mail: qinglai.wei@ia.ac.cn).

In recent years, inspired by deep learning techniques [26], the prevalent minibatch GD (MBGD) algorithms [27], [28] have been extended to TSK fuzzy system training to enhance computational efficiency. Unlike conventional GD algorithms, MBGD algorithms randomly select minibatches of samples for each iteration and use their computed gradients as unbiased estimators of the gradients of the full training set. These algorithms mitigate the stochastic instability inherent in stochastic GD (SGD) algorithms while simultaneously achieving an optimal trade-off between memory consumption and performance.

Furthermore, some studies have focused on improving the generalization performance of TSK fuzzy system learning. On the one hand, integrating adaptive parameter update strategies into recent algorithms can further enhance the learning performance of TSK fuzzy systems. Wu *et al.* [24] and Cui *et al.* [25] introduced the Adam [29] and AdaBound [30] algorithms into TSK fuzzy systems, respectively, to address the sensitivity of learning rates. In addition, Shi *et al.* [31] combined an MBGD algorithm with AdaBelief to design TSK fuzzy systems, which can perform well-generalizable end-to-end consistent feature extraction. On the other hand, integrating regularization optimization techniques is considered a promising approach. Wu *et al.* [24] proposed an MBGD algorithm incorporating regularization, DropRule, and AdaBound (MBGD-RDA), achieving state-of-the-art test accuracy on ten public machine learning datasets. Cui *et al.* [25] integrated uniform regularization and batch normalization into an MBGD algorithm to improve the generalization performance of TSK fuzzy systems in classification tasks. Cui *et al.* [32] proposed a layer normalization strategy to balance the rule activation intensity of TSK fuzzy systems, demonstrating improved model generalization performance across varying feature dimensions, dataset sizes, optimizers, and rule base sizes.

Although existing studies [24], [25], [31], [32] have conducted valuable explorations into improving optimization algorithms for TSK fuzzy systems, they still face challenges in terms of computational time cost and memory overhead. On the one hand, using backward propagation to compute gradients of the loss function for TSK fuzzy systems has been the mainstream paradigm. This optimization technique performs forward and backward propagation as two sequential computational stages, which may not be the most efficient approach. On the other hand, in existing studies, the updates of antecedent and consequent parameters are scaled by the inverse square root of the exponentially weighted average of historical gradients' squares, which undoubtedly requires additional memory. For instance, the MBGD-RDA algorithm [24] maintains two additional values for each fuzzy parameter (the first- and second-order moment estimates), thereby doubling the memory requirements.

To address the aforementioned issues, the study suggests new and promising solutions. The main innovations and contributions of this study can be summarized as follows:

- A novel minibatch adaptive optimization algorithm based on forward automatic differentiation (MBAO-FAD) was proposed to optimize TSK fuzzy systems. The proposed MBAO-FAD algorithm can construct a training pipeline

for TSK fuzzy systems without backward propagation, thereby fundamentally differing from traditional algorithms and offering increased computationally competitive.

- An efficient gradient estimation method based on matrix-free, Jacobian-vector products was proposed. This method requires only a single forward automatic differentiation pass during the forward propagation of TSK fuzzy systems to achieve unbiased estimation of gradients for antecedent and consequent parameters.
- A minibatch fuzzy parameter update strategy based on non-negative matrix factorization was proposed. By employing the generalized Kullback-Leibler divergence, the gradient estimation matrix of fuzzy parameters was decomposed into a product of two rank-1 matrices, eliminating the need to store first- and second-order moment estimates and thereby reducing the memory requirements for training TSK fuzzy systems.
- Compared with other algorithms, the proposed MBAO-FAD algorithm demonstrated superior performance in terms of optimization accuracy, training time, and memory overhead across nine real-world datasets from multiple application domains.

The rest of the manuscript is organized as follows. Section II presents a brief review of the TSK fuzzy system and MBGD algorithm. Section III introduces the proposed MBAO-FAD algorithm. Section IV presents our experimental results and discussion. Finally, Section V presents the conclusions and directions for future research.

II. BRIEF REVIEW

In this section, the TSK fuzzy system is first briefly reviewed, followed by the introduction of the MBGD algorithm for designing TSK fuzzy systems.

A. TSK Fuzzy System

The TSK fuzzy system primarily comprises four components: a fuzzifier, a rule base, an inference engine, and a defuzzifier. To begin, let the crisp input be $\mathbf{x} = (x_1, \dots, x_m, \dots, x_M)^T \in \mathbb{R}^{M \times 1}$.

1) *Fuzzifier*: The fuzzifier converts the crisp input $\mathbf{x}$ into a fuzzy set $\tilde{X}$. The Gaussian membership function can be used to characterize $\tilde{X}$ and can be expressed as

$$\mu_{\tilde{X}_{r,m}}(x_m) = \exp \left[-\frac{(x_m - c_{r,m})^2}{2\sigma_{r,m}^2} \right], \quad (1)$$

where $\mu_{\tilde{X}_{r,m}}$ is the membership degree, $r \in \{1, 2, \dots, R\}$, and $m \in \{1, 2, \dots, M\}$. $c_{r,m}$ and $\sigma_{r,m}$ are the mean and standard deviation of $\mu_{\tilde{X}_{r,m}}$, respectively.

2) *Rule Base*: The TSK fuzzy rule is determined by a polynomial associated with the input $\mathbf{x}$. The r -th TSK fuzzy rule can be expressed as

$$\begin{aligned} \text{IF } : x_1 \text{ is } \tilde{X}_{r,1} \text{ and } \dots \text{ and } x_M \text{ is } \tilde{X}_{r,M} \\ \text{THEN } : y_r(\mathbf{x}) = b_r + \sum_{m=1}^M \omega_{r,m} \cdot x_m, \end{aligned} \quad (2)$$

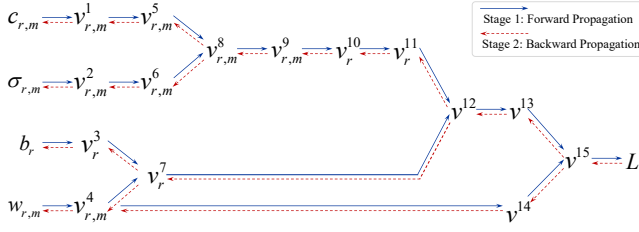


Fig. 1. Diagram of the two-stage calculation process for four types of fuzzy parameters in the MBGD algorithm. In the figure, the forward calculation process of the TSK fuzzy system is divided into 15 steps, as discussed in Section II-A and Table I. In backward propagation, the gradient of each intermediate variable v can be computed using the chain rule [24].

where b_r and $\omega_{r,m}$ are the biases and weights of the r -th TSK fuzzy rule, respectively.

3) *Inference Engine*: According to the antecedents of the TSK fuzzy rule, the t -product is commonly used to compute the firing strength for the r -th rule in the inference engine:

$$F_r(\mathbf{x}) = \prod_{m=1}^M \mu_{\tilde{x}_{r,m}}(x_m). \quad (3)$$

Then, F_r can be further normalized as

$$\bar{F}_r(\mathbf{x}) = \frac{F_r(\mathbf{x})}{\sum_{k=1}^R F_k(\mathbf{x})}. \quad (4)$$

4) *Defuzzifier*: The defuzzifier ultimately converts the fuzzy value into a crisp output y :

$$y(\mathbf{x}) = \sum_{r=1}^R \bar{F}_r(\mathbf{x}) \cdot y_r(\mathbf{x}). \quad (5)$$

B. MBGD Algorithm

According to Section II-A, the learnable fuzzy parameters must be optimized for designing an efficient TSK fuzzy system. These parameters, namely, $c \in \mathbb{R}^{M \times R}$, $\sigma \in \mathbb{R}^{M \times R}$, $\omega \in \mathbb{R}^{M \times R}$, and $b \in \mathbb{R}^R$, are usually expressed as the parameter vector θ . This is often considered as an optimization problem

$$\theta^* = \arg \min_{\theta} \|y(\mathbf{x}_n; \theta) - y_n\|, \quad (6)$$

where $y(\cdot; \theta)$ represents the designed fuzzy model between input $\mathbf{x}_n$ and output y_n , n represents the n -th sample in the training dataset $\mathcal{D} = \{\mathbf{x}_n, y_n\}_{n=1}^N$, and $n \in \{1, 2, \dots, N\}$.

Inspired by deep learning techniques, Wu *et al.* [24] introduced an MBGD algorithm into TSK fuzzy systems and successfully applied it to machine learning tasks in the context of big data. Such algorithms must typically solve machine learning tasks in two stages, as shown in Fig. 1.

1) *Forward Propagation*: N_{bs} (an integer between 1 and N) samples are obtained from the total training dataset; this subset is called a minibatch. Then, the output $y(\mathbf{x}_n; \theta)$ of the TSK fuzzy system on this minibatch is calculated and its accuracy is evaluated. This is usually achieved by designing a suitable loss function [24]:

$$L = \sum_{n=1}^{N_{bs}} [y_n - y(\mathbf{x}_n; \theta)]^2 + \frac{\lambda}{2} \sum_{r=1}^R \sum_{m=1}^M \omega_{r,m}^2, \quad (7)$$

Algorithm 1: Adaptive parameter update strategy in MBGD, taking Adam [25] as an example.

Require:

Initial learning rate α , first-moment decay β_1 , second-moment decay β_2 , regularization constant ϵ .

Initialize:

The parameters θ_0 of the antecedent and consequent of the TSK fuzzy system are initialized randomly;

The first- and second-moment estimates are set to 0, $A_0 \leftarrow 0$, $B_0 \leftarrow 0$.

Compute:

for $t = 1, 2, \dots, T$ **do**

$G_t \leftarrow \partial L / \partial \theta$ by derived gradient expression [24];

$A_t \leftarrow \beta_1 A_{t-1} + (1 - \beta_1) G_t$;

$B_t \leftarrow \beta_2 B_{t-1} + (1 - \beta_2) G_t^2$;

$\hat{A}_t \leftarrow \frac{A_t}{(1 - \beta_1^t)}$;

$\hat{B}_t \leftarrow \frac{B_t}{(1 - \beta_2^t)}$;

$\theta_t \leftarrow \theta_{t-1} - \alpha \frac{\hat{A}_t}{(\sqrt{\hat{B}_t} + \epsilon)}$.

end

Return: θ_T .

where the second term is the regularization term and λ is the normalized coefficient; $\lambda \geq 0$. This loss function can be expressed as $L(\theta) : \mathbb{R}^q \rightarrow \mathbb{R}$, where q represents the number of fuzzy parameters and $q = 2MN_m + (M + 1) \cdot (N_m)^M$.

2) *Backward Propagation*: The gradients $G \leftarrow \partial L / \partial \theta$ of all fuzzy parameters θ are calculated based on a manually derived gradient expression [24]. The negative gradient direction of the ∂L is evaluated on this minibatch. Subsequently, adaptive strategies for updating parameter θ of the TSK fuzzy system are introduced, including Adam [25] and AdaBound [24]. The detailed update procedure of Adam is presented in Algorithm 1.

The effectiveness of MBGD algorithms relies heavily on adaptive, gradient-based optimization. However, these algorithms continue to encounter challenges in the design of efficient TSK fuzzy systems. As a two-stage forward-backward algorithm, MBGD may potentially affect the computational complexity and time cost of the training pipeline of TSK fuzzy systems. This is because the MBGD algorithm must be converted into a serial computer program to accurately compute the gradient of L with respect to numerous parameters θ . In addition, these adaptive algorithms must store the historical square root of the gradients, i.e., first-order moment estimate A_t and second-order moment estimate B_t , as presented in Algorithm 1. This indicates that remarkable performance improvements also require improvements in the memory capacity required to store these parameters θ . In particular, these existing algorithms [24], [25], [31], [32] will undoubtedly double their memory footprint. TSK fuzzy systems inherently suffer from rule explosion, which further aggravates the situation.

TABLE I
COMPUTATIONAL PROCESS OF THE FOUR TYPES OF FUZZY PARAMETERS

Step	L	Step	$\frac{\partial L}{\partial c_{r,m}}$	$\frac{\partial L}{\partial \sigma_{r,m}}$	$\frac{\partial L}{\partial w_{r,m}}$	$\frac{\partial L}{\partial b_r}$
$v_{r,m}^1$	$c_{r,m}$	$\dot{v}_{r,m}^1$	1	0	0	0
$v_{r,m}^2$	$\sigma_{r,m}$	$\dot{v}_{r,m}^2$	0	1	0	0
$v_{r,m}^3$	b_r	$\dot{v}_{r,m}^3$	0	0	0	1
$v_{r,m}^4$	$w_{r,m}$	$\dot{v}_{r,m}^4$	0	0	1	0
$v_{r,m}^5$	$x_m - v_{r,m}^1$	$\dot{v}_{r,m}^5$	-1	0	0	0
$v_{r,m}^6$	$2(v_{r,m}^2)^2$	$\dot{v}_{r,m}^6$	0	$4v_{r,m}^2$	0	0
v_r^7	$v_r^3 + \sum_{m=1}^M x_m \times v_{r,m}^4$	$\dot{v}_r^7$	0	0	$\sum_{m=1}^M x_m \times \dot{v}_{r,m}^4$	$\dot{v}_r^3$
$v_{r,m}^8$	$\frac{-(v_{r,m}^5)^2}{v_{r,m}^6}$	$\dot{v}_{r,m}^8$	$\frac{2}{v_{r,m}^6} v_{r,m}^5$	$(\frac{v_{r,m}^5}{v_{r,m}^6})^2 \dot{v}_{r,m}^6$	0	0
$v_{r,m}^9$	$e^{v_{r,m}^8}$	$\dot{v}_{r,m}^9$	$v_{r,m}^9 \dot{v}_{r,m}^8$	$v_{r,m}^9 \dot{v}_{r,m}^8$	0	0
v_r^{10}	$\prod_{m=1}^M v_{r,m}^9$	$\dot{v}_r^{10}$	$\prod_{m=1, m \neq j}^M v_{r,m}^9 \dot{v}_{r,j}^9$	$\prod_{m=1, m \neq j}^M v_{r,m}^9 \dot{v}_{r,j}^9$	0	0
v_r^{11}	$\frac{v_r^{10}}{\sum_{k=1}^R v_k^{10}}$	$\dot{v}_r^{11}$	$\frac{\dot{v}_r^{10} \sum_{k=1}^R v_k^{10} - v_r^{10} \sum_{k=1}^R \dot{v}_k^{10}}{(\sum_{k=1}^R v_k^{10})^2}$	$\frac{\dot{v}_r^{10} \sum_{k=1}^R v_k^{10} - v_r^{10} \sum_{k=1}^R \dot{v}_k^{10}}{(\sum_{k=1}^R v_k^{10})^2}$	0	0
v_r^{12}	$\sum_{r=1}^R v_r^{11} v_r^7$	$\dot{v}_r^{12}$	$\sum_{r=1}^R \dot{v}_r^{11} v_r^7$	$\sum_{r=1}^R \dot{v}_r^{11} v_r^7$	$\sum_{r=1}^R v_r^{11} \dot{v}_r^7$	$\sum_{r=1}^R v_r^{11} \dot{v}_r^7$
v^{13}	$\frac{1}{2} \sum_{n=1}^{N_{bs}} (y_n - v^{12})^2$	$\dot{v}^{13}$	$\sum_{n=1}^{N_{bs}} (v^{12} - y_n) \dot{v}^{12}$	$\sum_{n=1}^{N_{bs}} (v^{12} - y_n) \dot{v}^{12}$	$\sum_{n=1}^{N_{bs}} (v^{12} - y_n) \dot{v}^{12}$	$\sum_{n=1}^{N_{bs}} (v^{12} - y_n) \dot{v}^{12}$
v^{14}	$\frac{\lambda}{2} \sum_{r=1}^R \sum_{m=1}^M (v_{r,m}^4)^2$	$\dot{v}^{14}$	0	0	$\lambda \dot{v}_{r,m}^4$	0
v^{15}	$v^{13} + v^{14}$	$\dot{v}^{15}$	$\dot{v}^{13}$	$\dot{v}^{13}$	$\dot{v}^{13} + \dot{v}^{14}$	$\dot{v}^{13}$

- setting $\dot{v}_{r,m}^1 = 1$, computing $\frac{\partial L}{\partial c_{r,m}}$; setting $\dot{v}_{r,m}^2 = 1$, computing $\frac{\partial L}{\partial \sigma_{r,m}}$.
- setting $\dot{v}_r^3 = 1$, computing $\frac{\partial L}{\partial b_r}$; setting $\dot{v}_{r,m}^4 = 1$, computing $\frac{\partial L}{\partial w_{r,m}}$.

III. MBAO-FAD

This section presents a detailed discussion on the application of the MBAO-FAD algorithm in the design of efficient TSK fuzzy systems. First, the gradient of the fuzzy parameter is estimated via forward automatic differentiation, eliminating the need for backward propagation and thereby reducing computation time. Second, a fuzzy parameter update strategy based on non-negative matrix factorization is proposed to reduce the memory overhead.

A. Forward Automatic Differentiation-Based Gradient Estimation

This study proposes a forward automatic differentiation-based gradient estimation method and demonstrates its unbiased estimation. Using this method, the values of intermediate variables and their derivatives with respect to fuzzy parameters are calculated concurrently in the same forward propagation process, completely eliminating the need for back propagation. This is completely different from the calculations shown in Fig. 1.

As presented in Table I, the original forward computation on the left is expanded via tangents on the right, with each row representing a direct expansion of the left-hand side. For computing the derivative of the loss function $L(\theta)$, the derivative of each intermediate variable $\dot{v}_i$ is computed and the derivative of each $\dot{v}_i$ is exclusively related to v_i in each computational operation. Applying the chain rule to each elementary operation in the forward primal trace, the derivative of each intermediate variable $\dot{v}_i$ are obtained from the computational operation in the forward original trajectory and the complete

tangents trajectory $\dot{v}_i$ are generated in the computational graph shown in Fig. 1. Then, the derivative of the last variable $\dot{v}_{15}$ is computed by synchronizing the intermediate v_i with their corresponding tangents $\dot{v}_i$.

The forward automatic differentiation can be generalized naturally to compute the Jacobian matrix $\mathcal{J}_L$ of the loss function $L(\theta)$. Thus, in the forward automatic differentiation, each forward propagation only requires setting one of the fuzzy parameters $\theta_i = 1$ and setting the rest to zero, as presented in Table I. Setting $\theta_i = e_i$ and computing the derivative of $L(\theta)$ at $\theta = a$ result in

$$\dot{L}_i = \left. \frac{\partial L}{\partial \theta_i} \right|_{\theta=a, \dot{\theta}=e_i}, \quad (8)$$

where e_i is the i -th unit vector.

In each forward propagation, only one element of the Jacobian matrix is computed. The complete Jacobian matrix $\mathcal{J}_L$ requires q forward propagations because there are q fuzzy parameters:

$$\mathcal{J}_L = \left[\frac{\partial L}{\partial \theta_1}, \dots, \frac{\partial L}{\partial \theta_q} \right] \Big|_{\theta=a}. \quad (9)$$

As mentioned earlier, the effectiveness of computing the Jacobian matrix is directly influenced by the number of fuzzy parameters, especially in cases with (i) an excessive number of fuzzy sets in the partitions or (ii) highly complex rules.

Hence, an efficient gradient estimation method based on matrix-free Jacobian-vector products is proposed for further

accelerating forward automatic differentiation. The Jacobian-vector products can be expressed as

$$\mathcal{J}_L s = \left[\frac{\partial L}{\partial \theta_1}, \dots, \frac{\partial L}{\partial \theta_q} \right] \begin{bmatrix} s_1 \\ \vdots \\ s_q \end{bmatrix}, \quad (10)$$

where $s \sim p(s)$ is a perturbation vector following a multivariate distribution, and $s \in \mathbb{R}^q$. The components of the perturbation vector s are mutually independent while satisfying the condition of zero mean and unit variance, i.e., $s \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.

Thus, the Jacobian-vector product can be computed in only one forward propagation by simple initializing with $\dot{\theta} = s$. In this case, the directional derivative along a given vector s can be obtained as a linear combination of the partial derivatives. This is the projection of the gradient of ∇L at the point θ onto the given direction vector s , representing the gradient along this given direction

$$\nabla L(\theta) \cdot s = \sum_{i=1}^q \frac{\partial L}{\partial \theta_i} s_i. \quad (11)$$

Hence, the gradient $\mathcal{G}(\theta)$ of fuzzy parameters can be estimated by multiplying the directional derivative $\nabla L(\theta) \cdot s$ with the perturbation vector s as follows:

$$\mathcal{G}(\theta) = \nabla L(\theta) \cdot s \cdot s = \begin{bmatrix} \frac{\partial L}{\partial \theta_1} s_1^2 + \frac{\partial L}{\partial \theta_2} s_1 s_2 + \dots + \frac{\partial L}{\partial \theta_q} s_1 s_q \\ \frac{\partial L}{\partial \theta_1} s_1 s_2 + \frac{\partial L}{\partial \theta_2} s_2^2 + \dots + \frac{\partial L}{\partial \theta_q} s_2 s_q \\ \vdots \\ \frac{\partial L}{\partial \theta_1} s_1 s_q + \frac{\partial L}{\partial \theta_2} s_2 s_q + \dots + \frac{\partial L}{\partial \theta_q} s_q^2 \end{bmatrix} \quad (12)$$

For $\mathcal{G}(\theta)$, we have the following result.

Theorem 1. The forward gradient $\mathcal{G}(\theta)$ is an unbiased estimator of the gradient $\nabla L(\theta)$.

Proof. Based on (12), $\forall \mathcal{G}_i(\theta)$ has the following form:

$$\mathcal{G}_i(\theta) = \frac{\partial L}{\partial \theta_i} s_i^2 + \sum_{j \neq i} \frac{\partial L}{\partial \theta_j} s_i s_j. \quad (13)$$

Then,

$$\begin{aligned} \mathbb{E}[\mathcal{G}_i(\theta)] &= \mathbb{E} \left[\frac{\partial L}{\partial \theta_i} s_i^2 + \sum_{j \neq i} \frac{\partial L}{\partial \theta_j} s_i s_j \right] \\ &= \mathbb{E} \left[\frac{\partial L}{\partial \theta_i} s_i^2 \right] + \mathbb{E} \left[\sum_{j \neq i} \frac{\partial L}{\partial \theta_j} s_i s_j \right] \\ &= \mathbb{E} \left[\frac{\partial L}{\partial \theta_i} s_i^2 \right] + \sum_{j \neq i} \mathbb{E} \left[\frac{\partial L}{\partial \theta_j} s_i s_j \right] \\ &= \frac{\partial L}{\partial \theta_i} \mathbb{E}[s_i^2] + \sum_{j \neq i} \frac{\partial L}{\partial \theta_j} \mathbb{E}[s_i s_j], \end{aligned} \quad (14)$$

where $\mathbb{E}[\cdot]$ represent the expected value of the corresponding components.

Because $s \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, s_i and s_j are two independent and identically distributed variables. Thus, the following conclusions can be obtained.

Theorem 2. For the first part in (14), we have $\mathbb{E}[s_i^2] = 1$ when $\mathbb{E}[s_i] = 0$ and $\text{Var}[s_i] = 1$.

Proof. For the Variance, we can obtain

$$\begin{aligned} \text{Var}[s_i] &= \mathbb{E}[(s_i - \mathbb{E}[s_i])^2] \\ &= \mathbb{E}[s_i^2] - \mathbb{E}[s_i]^2. \end{aligned} \quad (15)$$

then,

$$\begin{aligned} \mathbb{E}[s_i^2] &= \mathbb{E}[s_i]^2 + \text{Var}[s_i] \\ &= 1. \end{aligned} \quad (16)$$

Theorem 3. For the second part in (14), we have $\mathbb{E}[s_i s_j] = 0$ when $\mathbb{E}[s_i] = 0$ or $\mathbb{E}[s_j] = 0$.

Proof. Since s_i and s_j are two independent and identically distributed variables, the expected value is

$$\begin{aligned} \mathbb{E}[s_i s_j] &= \mathbb{E}[s_i] \mathbb{E}[s_j] \\ &= 0. \end{aligned} \quad (17)$$

Now, the (14) can be reduced as

$$\mathbb{E}[\mathcal{G}_i(\theta)] = \frac{\partial L}{\partial \theta_i}, \quad (18)$$

therefore,

$$\mathbb{E}[\mathcal{G}(\theta)] = \nabla L(\theta). \quad (19)$$

Now, the gradient $\mathcal{G}(\theta)$ values of fuzzy parameters are estimated in only one forward propagation by simply choosing a satisfaction perturbation vector s and initializing with $\dot{\theta} = s$. The loss L of the TSK fuzzy system and its Jacobian-vector product must be simultaneously evaluated. Evaluating the Jacobian-vector product does not require computing the Jacobian matrix $\mathcal{J}_L$. This can considerably accelerate forward automatic differentiation.

B. Parameter Update Strategy Based on Non-Negative Matrix Factorization

Existing adaptive parameter update strategies in MBGD algorithms must store the gradients of fuzzy parameters and their first-order moment estimation A_t and second-order moment estimation B_t , as presented in Algorithm 1. This imposes substantial additional memory overhead, particularly in the context of increasingly large-scale data. To solve this problem, a memory-efficient adaptive strategy based on non-negative matrix factorization is proposed to update the parameters $\theta \in \mathbb{R}^{M \times R}$ of TSK fuzzy systems.

In each iteration, the gradients $\mathcal{G}$ of fuzzy parameters on the current minibatch are evaluated using the forward automatic differentiation technique described in Section III-A. Subsequently, the second-order moment estimates B_t of the $\mathcal{G}$ are approximated by maintaining exponential moving averages of two low-rank factors, denoted as w_t and h_t , through non-negative matrix factorization. Specifically, the approximation is achieved by minimizing the Kullback–Leibler divergence between the second-order moment estimates and the factorized matrices w_t and h_t , formulated as follows:

$$w_t = B_t \mathbf{1}_R, h_t = \frac{\mathbf{1}_M^T B_t}{\mathbf{1}_M^T B_t \mathbf{1}_R}, \quad (20)$$

where $1_R = (1, \dots, 1) \in \mathbb{R}^R$ represents a $1 \times R$ dimension unit column vector. This approach requires storing only the moving averages of these low-rank factors instead of the full second-order moment estimates B_t , thereby reducing the memory complexity from $\mathcal{O}(rm)$ to $\mathcal{O}(r + m)$. The second-order moment estimate can now be expressed as

$$B_t = \frac{w_t h_t}{1_M^T w_t}. \quad (21)$$

The unscaled update is then calculated based on the approximate second-order moment estimate

$$u_t = \frac{\mathcal{G}_t}{\sqrt{B_t}}. \quad (22)$$

To prevent excessively large update steps that may lead to high approximation errors, updates are only generated when the root mean square of all components in u_t exceeds a predefined threshold d .

$$\hat{u}_t = \frac{u_t}{\max(1, RMS(u_t)/d)}, \quad (23)$$

where $RMS(\cdot)$ is root the mean square function. Subsequently, the first-order moment estimate A_t is calculated based on $\hat{u}_t$, which is similar to Adam:

$$A_t = \beta_1 A_{t-1} + (1 - \beta_1) \hat{u}_t, \quad (24)$$

Based on empirical observations, when the discrepancy between A_t and $\hat{u}_t$ becomes large, the optimization trajectory tends to increasingly deviate from the desired direction, often causing substantial performance degradation. To mitigate this issue, an instability matrix q_t is introduced to quantitatively evaluate the degree of such deviation, explicitly capturing the bias induced by the approximate updates:

$$q_t = (\hat{u}_t - A_t)^2. \quad (25)$$

To avoid the additional memory overhead required for storing q_t , the instability matrix q_t is approximated by maintaining exponential moving averages of two new low-rank factors, denoted as W_t and H_t , in a manner analogous to (20).

$$W_t = q_t 1_R, H_t = \frac{1_M^T q_t}{1_M^T q_t 1_R}. \quad (26)$$

The instability matrix q_t is decomposed into W_t and H_t as follows:

$$q_t = \frac{W_t H_t}{1_M^T W_t}. \quad (27)$$

In the proposed adaptive fuzzy parameter updating strategy, the instability matrix q_t serves as the denominator of the original first-order moment estimate A_t , enabling more adaptive parameter updates.

$$\theta_t = \theta_{t-1} - \alpha \frac{A_t}{\sqrt{q_t}}, \quad (28)$$

where, α is the initial learning rate. According to the aforementioned equation, when A_t and $\hat{u}_t$ are considerably far apart (i.e., when the instability matrix q_t is large), $1/\sqrt{q_t}$ is small, indicating that the confidence of the first-order moment estimate A_t is low and that a small optimization step should be performed. Conversely, if A_t and $\hat{u}_t$ are in close agreement,

Algorithm 2: Training a TSK fuzzy system using the proposed MBAO–FAD.

Require:

Training dataset $\mathcal{D}$, loss function L , regularization parameters λ , DropRule rate P [24], the number of Gaussian MFs for each input N_m , the maximum number of training iterations T , the minibatch size N_{bs} , and the initial hyperparameter of MBAO–FAD, $\alpha, \epsilon, \epsilon_1, \epsilon_2, \beta_1, \beta_2, \beta_3, d$.

Initialize:

The parameters θ_0 of the antecedent and consequent of the TSK fuzzy system are initialized randomly;
 $A_0 \leftarrow 0, W_0 \leftarrow 0, H_0 \leftarrow 0, w_0 \leftarrow 0, h_0 \leftarrow 0$;

Compute:

for $t = 1, 2, \dots, T$ **do**

Gain minibatch $\mathcal{B} = \{\mathbf{x}_n, y_n\}_{n=1}^{N_{bs}}$ from $\mathcal{D}$;
 $s_t \sim \mathcal{N}(\theta, I)$;
 $L_t, d_t \leftarrow L(\theta_{t-1}), \nabla L(\theta_{t-1}) \cdot s_t$;
 $\mathcal{G}_t \leftarrow s_t \cdot d_t$;
 $w_t \leftarrow \beta_2 w_{t-1} + (1 - \beta_2)(\mathcal{G}_t^2 + \epsilon_1 1_M 1_R^T) 1_R$;
 $h_t \leftarrow \beta_2 h_{t-1} + (1 - \beta_2) 1_M^T (\mathcal{G}_t^2 + \epsilon_1 1_M 1_R^T)$;
 $B_t \leftarrow w_t h_t / 1_M^T w_t$;
 $u_t \leftarrow \mathcal{G}_t / \sqrt{B_t}$;
 $\hat{u}_t \leftarrow u_t / \max(1, RMS(u_t)/d)$;
 $A_t \leftarrow \beta_1 A_{t-1} + (1 - \beta_1) \hat{u}_t$;
 $q_t \leftarrow (\hat{u}_t - A_t)^2$;
 $W_t \leftarrow \beta_3 W_{t-1} + (1 - \beta_3)(q_t + \epsilon_2 1_M 1_R^T) 1_R$;
 $H_t \leftarrow \beta_3 H_{t-1} + (1 - \beta_3) 1_M^T (q_t + \epsilon_2 1_M 1_R^T)$;
 $\theta_t \leftarrow \theta_{t-1} - \alpha \frac{A_t}{\sqrt{W_T H_t / 1_M^T W_t}}$;

end

Return: θ_T .

a larger optimization step can be performed. Because q_t is decomposed into W_t and H_t , the aforementioned expression can be rewritten as

$$\theta_t = \theta_{t-1} - \alpha \frac{A_t}{\sqrt{W_T H_t / 1_M^T W_t}}. \quad (29)$$

C. Proposed MBAO–FAD Algorithm

To maximize computational efficiency and minimize memory overhead, we develop an MBAO–FAD algorithm, a novel training pipeline, for efficiently designing TSK fuzzy systems. The corresponding pseudocode is presented in Algorithm 2. For each minibatch $\mathcal{B}$ during iteration, MBAO–FAD executes two core training operations:

1) *Estimation of the Gradient in Forward Propagation:*

As detailed in Section III–A, the loss L of the TSK fuzzy system and its gradients $\partial L / \partial \theta$ with respect to all antecedent and consequent parameters θ are computed simultaneously in a single forward pass. This is achieved through our proposed forward-mode automatic differentiation-based gradient estimation method, which entirely eliminates the need for backpropagation, thereby considerably reducing computational time complexity.

TABLE II
SUMMARY OF THE NINE REGRESSION DATASETS [24]

Dataset	Source	No. of examples	No. of raw features	No. of used features	No. of model parameters
NO2	StatLib	500	7	7	1052
Housing	UCI	506	13	9	5156
Concrete	UCI	1030	8	8	2336
Airfoil	UCI	1503	5	5	212
Wine-Red	UCI	1599	11	6	472
Abalone	UCI	4177	8	7	1052
Wine-White	UCI	4898	11	6	472
PowerPlant	UCI	9568	4	4	96
Protein	UCI	45730	9	9	5156

TABLE III
DETAILED CONFIGURATION OF ALL COMPARATIVE METHODS

Algorithm	N_m	epoch	P	N_{bs}	α	ϵ	ϵ_1	ϵ_2	β_1	β_2	β_3	γ	d
MBGD-Adam	2	500	0.7	64	$1e^{-3}$	$1e^{-8}$	/	/	0.9	0.999	/	/	/
MBGD-RDA	2	500	0.7	64	$1e^{-3}$	$1e^{-8}$	/	/	0.9	0.999	/	/	/
MBGD-AdaGrad	2	500	0.7	64	$1e^{-3}$	$1e^{-8}$	/	/	/	/	/	/	/
MBGD-SGD	2	500	0.7	64	$1e^{-3}$	/	/	/	/	/	/	/	/
MBGD-RMSP	2	500	0.7	64	$1e^{-3}$	$1e^{-8}$	/	/	/	/	/	0.9	/
MBGD-Momentum	2	500	0.7	64	$1e^{-3}$	/	/	/	/	/	/	0.9	/
MBAO-FAD	2	500	0.7	64	$1e^{-3}$	/	$1e^{-30}$	$1e^{-16}$	0.9	0.999	0.99999	/	1

2) *Adaptive Update of Fuzzy Parameters*: As described in Section III-B, all antecedent and consequent parameters θ are adaptively updated based on the forward gradient estimates $\mathcal{G}$ from the previous step. MBAO-FAD incorporates a memory-efficient fuzzy parameter update strategy that obviates the substantial storage requirements for second-moment estimates B_t via non-negative matrix factorization.

The PyTorch implementation of MBAO-FAD is publicly available at <https://github.com/hzhangSDU/MBAO-FAD>.

IV. EXPERIMENTS AND DISCUSSIONS

This section presents comparative experiments and performance analysis of the proposed MBAO-FAD algorithm. First, nine commonly used public datasets in machine learning are briefly introduced. Subsequently, detailed experimental configurations for the MBAO-FAD and its comparative algorithms are presented. Finally, the performance, including the optimization accuracy, computational time overhead, and memory overhead, are analyzed and discussed in detail.

A. Brief Introduction of the Nine Applied Datasets

The nine regression datasets are from the CMU StatLib Datasets Archive ¹ and UCI MachineLearning Knowledge Base ². The detailed information is listed in Table II.

B. Comparative Algorithms and Detailed Configurations

The detailed experimental configurations for MBAO-FAD and its comparative algorithms are listed in Table III. Adaptive MBGD algorithms, including MGBD-Adam, MGBD-RDA, and MGBD-AdaGrad are selected as comparison methods. To comprehensively verify the effectiveness and advantages of the proposed algorithm, several nonadaptive MBGD algorithms—including MGBD-SGD, MGBD-RMSProp (MGBD-RMSP),

and MGBD-Momentum are also selected for comparisons. All the comparative algorithms are implemented using the PyTorch framework and executed with forward and backward propagation for the same number of iterations. All experiments are conducted on a laptop equipped with a 13-th generation Intel®Core™ i9-13900KS processor, 16GB of RAM, and an NVIDIA RTX2090 GPU.

C. Training and Testing Performance

The iterative loss (testing RMSEs) for all algorithms is illustrated in Fig. 2, and the detailed training and testing RMSE metrics are listed in Table IV. As a novel algorithm without backward propagation, the proposed MBAO-FAD demonstrates the best predictive performance on seven datasets (i.e., NO2, Housing, Concrete, Airfoil, Wine-Red, PowerPlant, and Protein), achieving the lowest testing RMSEs. Although MBAO-FAD may not perform optimally on all datasets, it still attains testing RMSEs comparable with those of the best backward propagation-based algorithms and even outperforms adaptive MBGD algorithms, including MBGD-RDA and MBGD-Adam, on most datasets.

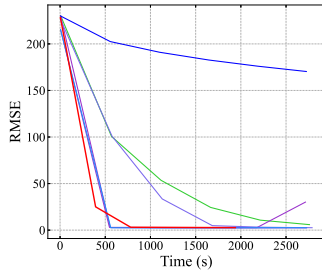
Furthermore, Fig. 2 and Table IV indicate that not all adaptive optimization algorithms can effectively train TSK fuzzy systems. For instance, the convergence speed of the MBGD-AdaGrad algorithm (indicated by the deep blue dashed line in Fig. 2) is notably slow among all algorithms, and its testing RMSE is even lower than that of the MBGD-SGD algorithm. These results indicate that gradient-based backward propagation algorithms, if flawed in updating learning rates, directly yield decreased TSK fuzzy system learning performance. However, the RMSE metrics across the nine datasets demonstrate that the proposed MBAO-FAD algorithm adapts and accurately updates the learning rates, thereby achieving outstanding learning performance.

¹[Online]. Available: <http://lib.stat.cmu.edu/datasets/>

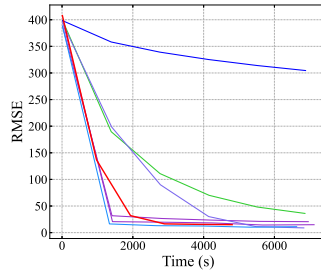
²[Online]. Available: <http://archive.ics.uci.edu/ml/index.php>

TABLE IV
TRAINING AND TESTING RMSES OF ALL COMPARATIVE ALGORITHMS ON NINE DATASETS

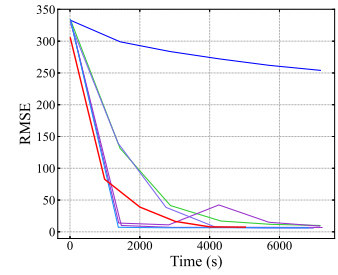
Datasets		MBGD-Adam	MBGD-RDA	MBGD-AdaGrad	MBGD-SGD	MBGD-RMSP	MBGD-Momentum	MBAO-FAD
NO2	training	12.2958	8.4988	82.8337	27.4482	8.4541	8.7042	7.4482
	testing	5.8928	2.7732	170.3597	30.1049	3.0744	2.7674	2.6903
Housing	training	22.2947	19.9763	90.4305	20.6092	29.2156	20.2693	29.3293
	testing	36.0984	15.8899	304.5627	20.5890	18.8902	16.1474	14.9938
Concrete	training	16.3422	20.4414	69.7122	21.1880	16.7845	20.8614	27.1150
	testing	9.9790	7.8845	254.0599	9.0637	7.8763	7.3970	7.3799
Airfoil	training	2.9837	2.5526	19.6740	3.1608	2.9370	2.4918	2.2183
	testing	10.2299	9.8246	80.5693	13.2083	10.7569	10.4362	8.8810
Wine-Red	training	3.9235	3.8910	24.5379	4.2304	2.9464	3.9456	3.6676
	testing	9.4894	7.1140	169.5382	7.8490	7.6672	7.2938	6.8957
Abalone	training	10.5009	11.4621	58.1754	11.3309	15.2644	11.6045	10.0133
	testing	39.8627	17.4045	991.4932	19.5988	19.9110	16.5985	16.6315
Wine-White	training	3.0775	3.7708	21.8049	4.2678	2.6188	3.9121	3.5008
	testing	17.1727	15.4947	376.3812	17.1283	17.2387	17.9757	17.0319
PowerPlant	training	4.2055	3.7241	15.5770	3.5070	3.4459	3.6997	1.9233
	testing	127.6766	135.1371	666.5887	122.2615	128.1777	138.2603	77.8137
Protein	training	2.5405	2.1184	17.8700	3.0042	2.2435	2.1632	1.5017
	testing	201.5498	177.3267	3339.540	236.1061	267.3858	197.0655	166.7901



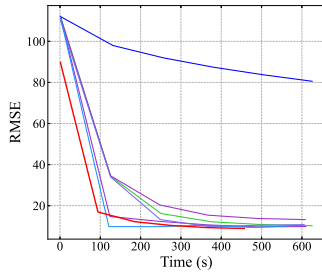
(a) NO2



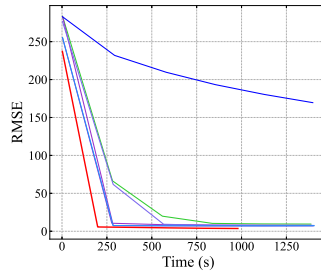
(b) Housing



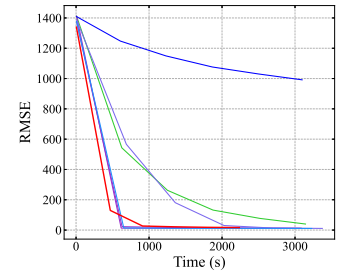
(c) Concrete



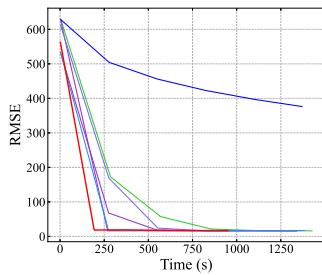
(d) Airfoil



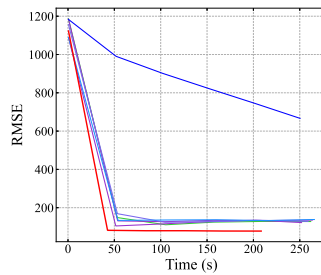
(e) Wine Red



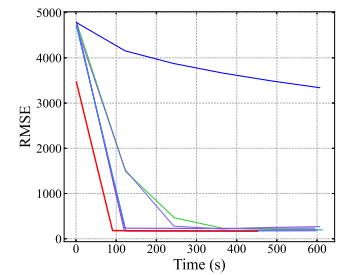
(f) Abalone



(g) Wine White



(h) PowerPlant



(i) Protein

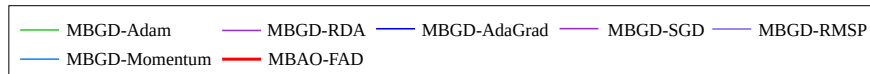


Fig. 2. Comparison of RMSEs between the proposed MBAO-FAD and other MBGD algorithms.

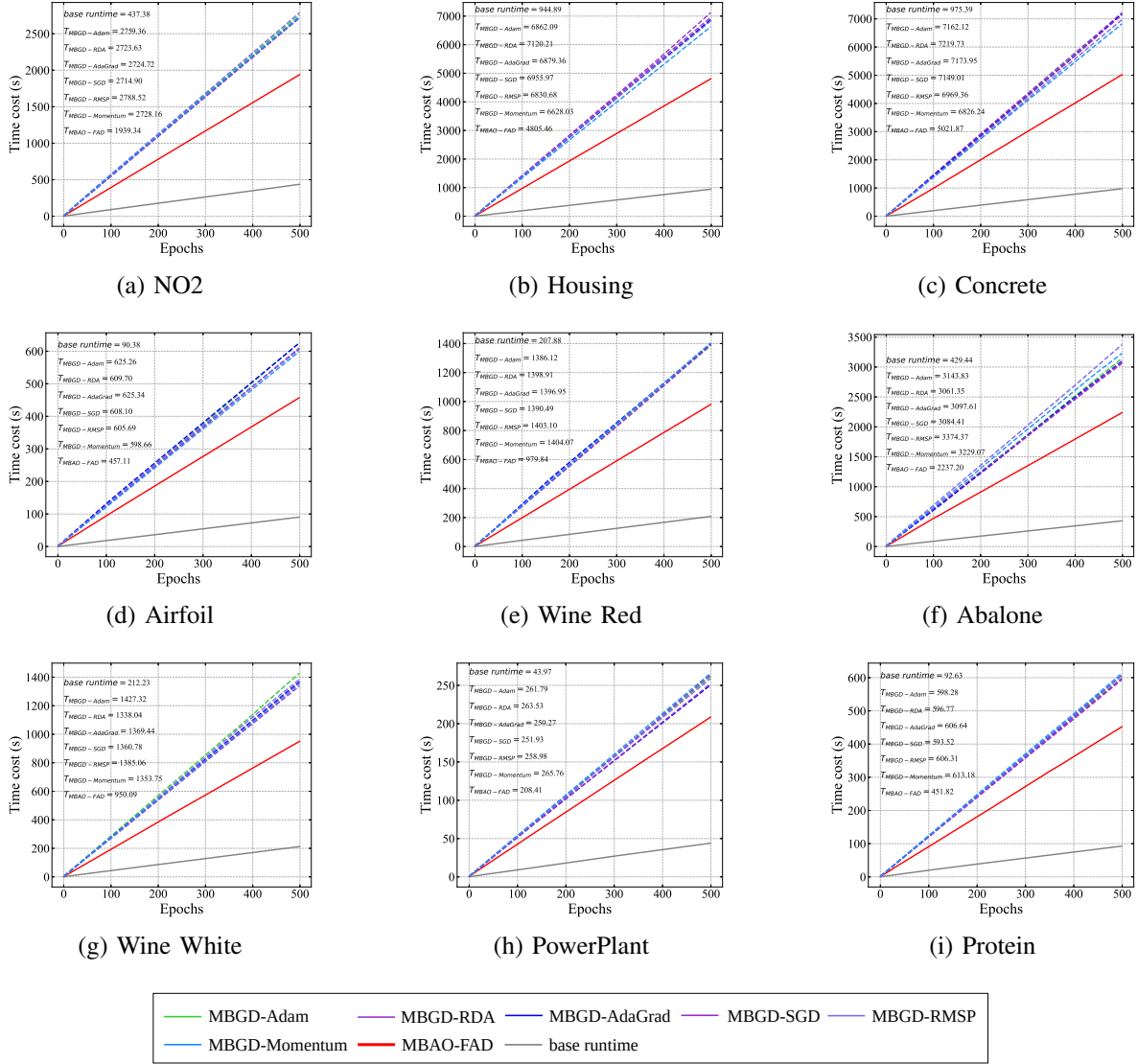


Fig. 3. Computational time cost between the proposed MBAO-FAD and its comparative algorithms.

D. Analysis of Computational Time Cost

As shown in Fig. 2, the proposed MBAO-FAD achieves the fastest convergence speed on all datasets. This is primarily because MBAO-FAD does not require backward propagation and only requires a single forward propagation. To more accurately measure the computational complexity using proposed MBAO-FAD, three metrics are introduced: computational time, computational efficiency, and extra time consumption, along with a more detailed analysis.

In practice, MBGD-based algorithms often incur a constant factor of computational overhead relative to the time required for forward computation (referring herein to the process of computing outputs based solely on given inputs without backward parameter optimization). However, relevant measurements for TSK fuzzy systems have not yet been conducted. To further accurately evaluate the improvement in computational efficiency achieved by the proposed MBAO-FAD compared with the other comparative algorithms, the following metrics

are defined: *runtime*, representing the time for the forward computation of TSK fuzzy systems (i.e., the base runtime); T , representing the computational time consumed by each algorithm to reach the final minimum RMSE loss through iterations (including the base runtime); and R , which evaluates the actual computational efficiency of different algorithms, satisfying the relationship $T = R \times \text{runtime}$. $\bar{R}$ is used to measure the additional computational cost of each method relative to that of the proposed MBAO-FAD. For example, the additional cost incurred by MBGD-RDA compared with the proposed MBAO-FAD is given by

$$\bar{R}_{\text{MBGD-RDA}} = \frac{T_{\text{MBGD-RDA}} - T_{\text{MBAO-FAD}}}{T_{\text{MBAO-FAD}}} \times 100\%. \quad (30)$$

Fig. 3 presents the T metric and *runtime* for all algorithms, showing that the proposed MBAO-FAD algorithm exhibits a considerable improvement in computational efficiency compared with other backward propagation-based algorithms such

TABLE V
THE DETAILED COMPUTATIONAL TIME COST BETWEEN THE PROPOSED MBAO-FAD AND ITS COMPARATIVE ALGORITHMS

Datasets		MBGD-Adam	MBGD-RDA	MBGD-AdaGrad	MBGD-SGD	MBGD-RMSP	MBGD-Momentum	MBAO-FAD
NO2	T	2759.36	2723.63	2724.72	2714.90	2788.52	2728.16	1939.34
	R	6.31	6.23	6.23	6.21	6.38	6.24	4.43
	$\vec{R}$	42.28%↑	40.44%↑	40.50%↑	39.99%↑	43.79%↑	40.67%↑	/
Housing	T	6862.09	7120.21	6879.36	6955.97	6830.68	6628.03	4805.46
	R	7.26	7.54	7.28	7.36	7.23	7.01	5.09
	$\vec{R}$	42.80%↑	48.17%↑	43.16%↑	44.75%↑	42.14%↑	37.93%↑	/
Concrete	T	7162.12	7219.73	7173.95	7149.01	6969.36	6826.24	5021.87
	R	7.34C	7.40	7.35	7.33	7.15	6.43	5.15
	$\vec{R}$	42.62%↑	43.77%↑	42.85%↑	42.36%↑	38.78%↑	35.93%↑	/
Airfoil	T	625.26	609.70	625.34	608.10	605.69	598.66	457.11
	R	6.92	6.75	6.92	6.73	6.70	6.62	5.06
	$\vec{R}$	36.78%↑	33.38%↑	36.80%↑	33.03%↑	32.50%↑	30.97%↑	/
Wine-Red	T	1386.12	1398.91	1396.95	1390.49	1403.10	1404.07	979.84
	R	6.67	6.73	6.72	6.69	6.75	6.75	4.71
	$\vec{R}$	41.46%↑	42.77%↑	42.57%↑	41.91%↑	43.20%↑	43.29%↑	/
Abalone	T	3143.83	3061.35	3097.61	3084.41	3374.37	3229.07	2237.20
	R	7.32	7.13	7.21	7.18	7.86	7.52	5.21
	$\vec{R}$	40.52%↑	36.84%↑	38.46%↑	37.87%↑	50.83%↑	44.34%↑	/
Wine-White	T	1427.32	1338.04	1369.44	1360.78	1385.06	1353.75	950.09
	R	6.73	6.30	6.45	6.41	6.53	6.38	4.48
	$\vec{R}$	50.23%↑	40.83%↑	44.14%↑	43.23%↑	45.78%↑	42.49%↑	/
PowerPlant	T	261.79	263.53	250.27	251.93	258.98	265.76	208.41
	R	5.95	5.99	5.69	5.73	5.89	6.04	4.74
	$\vec{R}$	25.62%↑	26.45%↑	20.09%↑	20.89%↑	24.27%↑	27.52%↑	/
Protein	T	598.28	596.77	606.64	593.52	606.31	613.18	451.82
	R	6.46	6.44	6.55	6.41	6.55	6.62	4.88
	$\vec{R}$	32.42%↑	32.08%↑	34.27%↑	31.36%↑	34.19%↑	35.71%↑	/

- T represents all of the computational time cost, and $T = R \times runtime$.
- $\vec{R}$ represents the additional computational time cost of each method relative to MBAO-FAD.
- $\uparrow$ represents the required additional computational time cost compared to MBAO-FAD.

as MBGD-RDA: the red solid line representing the MBAO-FAD algorithm is distinctly different from the dashed lines representing other MBGD algorithms. Additionally, Fig. 3 shows that for the same number of iterations, as the input dimension increases, the computational time expense T increases for all algorithms—including the proposed MBAO-FAD and comparative algorithms. Moreover, the larger the input dimension, the greater the difference in computational time expense between MBAO-FAD and other comparative algorithms, indicating a more pronounced advantage in time expense for the proposed MBAO-FAD algorithm. For example, on the PowerPlant dataset, MBGD-RDA requires an additional 26.45% training time expense compared with MBAO-FAD. Moreover, on the Housing dataset, the additional training time between MBGD-RDA and MBAO-FAD can increase to 48.17%.

Computational complexity measurement metrics in the comparative experiments are listed in Table V. In Table V, the relationship between the total computational expense of the TSK fuzzy system and the base runtime is presented, which also represents one of the contributions of this study. In the experiments conducted on the nine datasets, the computational expense of backward propagation-based MBGD algorithms in reaching the minimum RMSE loss is approximately $6 \sim 7$ times the base *runtime*, whereas the proposed MBAO-FAD requires only approximately $4 \sim 5$ times.

E. Analysis of Computational Memory Overhead

Calculating the memory overhead is crucial for evaluating the efficiency of different optimization algorithms. Table VI presents the computational memory overhead of the proposed MBAO-FAD algorithm and its comparative algorithms. In this study, the parameter count of the TSK fuzzy system considers only the influence of input dimensions M and the number of membership functions N_m . A complete rule base of TSK the fuzzy system is adopted, where the number of rules $N_R = N_m^M$ is determined by input dimensions M and N_m , thus disregarding the impact of rule count on the overall parameter count. As presented in Table VI, under the same input dimensions (with all N_m set to 2), the computational memory overhead of each algorithm remains consistent across different datasets.

The proposed MBAO-FAD algorithm has been optimized to reduce the computational memory overhead. Specifically, through the efficient non-negative matrix factorization, the parameter requirements for the second-order moment estimation can be effectively reduced, thus cutting down the total computational memory overhead. As indicated in Table VI, current baseline algorithms—MBGD-RDA and MBGD-Adam—both incur the highest memory consumption among adaptive MBGD algorithms. However, the proposed MBAO-FAD algorithm requires the least memory overhead. Specifically, across all nine datasets, the proposed MBAO-FAD algorithm achieved memory savings of 23.45%, 26.27%, 25.14%, 16.41%, 20.77%, 23.45%, 20.77%, 9.25%, and 26.27%, re-

TABLE VI
DETAILED COMPUTATIONAL MEMORY OVERHEAD BETWEEN THE PROPOSED MBAO-FAD AND ITS COMPARATIVE ALGORITHMS

Datasets		MBGD-Adam	MBGD-RDA	MBGD-AdaGrad	MBGD-SGD	MBGD-RMSP	MBGD-Momentum	MBAO-FAD
NO2	C	3160	3161	2106	1053	2106	2106	2419
	C_1	1052	1052	1052	/	1052	1052	1052
	C_2	1052	1052	/	/	/	/	308
Housing	C	15472	15473	10314	5157	10314	10314	11407
	C_1	5156	5156	5156	/	5156	5156	5156
	C_2	5156	5156	/	/	/	/	1088
Concrete	C	7012	7013	4674	2337	4674	4674	5249
	C_1	2336	2336	2336	/	2336	2336	2336
	C_2	2336	2336	/	/	/	/	570
Airfoil	C	640	641	426	213	426	426	535
	C_1	212	212	212	/	212	212	212
	C_2	212	212	/	/	/	/	104
Wine-Red	C	1420	1421	946	473	946	946	1125
	C_1	472	472	472	/	472	472	472
	C_2	472	472	/	/	/	/	174
Abalone	C	3160	3161	2106	1053	2106	2106	2419
	C_1	1052	1052	1052	/	1052	1052	1052
	C_2	1052	1052	/	/	/	/	308
Wine-White	C	1420	1421	946	473	946	946	1125
	C_1	472	472	472	/	472	472	472
	C_2	472	472	/	/	/	/	174
PowerPlant	C	292	293	194	97	194	194	265
	C_1	96	96	96	/	96	96	96
	C_2	96	96	/	/	/	/	66
Protein	C	15472	15473	10314	5157	10314	10314	11407
	C_1	5156	5156	5156	/	5156	5156	5156
	C_2	5156	5156	/	/	/	/	1088

- C represents all variables of different algorithms that need to be stored.
- C_1 represents the memory required for the first-order moment estimation.
- C_2 represents the memory required for the second-order moment estimation.
- / means that this item does not need to be stored.

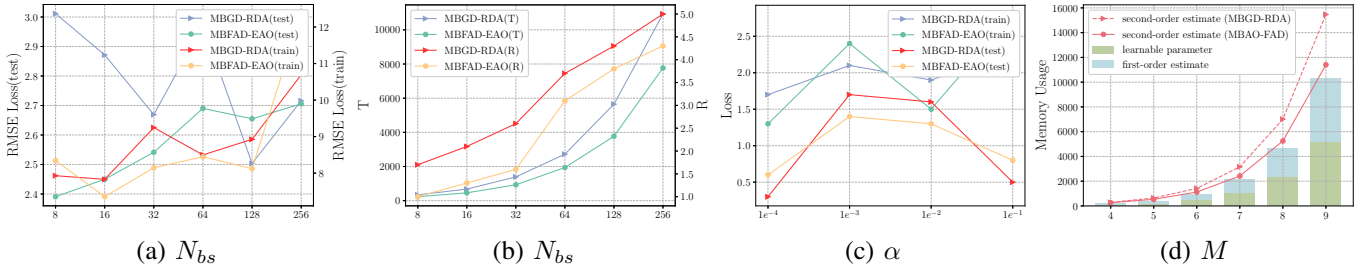


Fig. 4. Parameter sensitivity results for the proposed MBAO-FAD and comparison MBGD-RDA. All experiments are conducted on the PM2 dataset, with other hyperparameters configured according to the settings in Table III. (a) The impact of different mini-batch sizes N_{bs} on training and testing RMSE losses. (b) The effect of different N_{bs} values on computational time cost T and R , where $T = R \times runtime$. (c) The influence of different learning rates α on training and testing RMSE losses. (d) The effect of different input dimensions M on computational memory overhead during the training process.

spectively, compared with MBGD-RDA [24].

Furthermore, owing to the retention of relevant state variables for approximating the second-order moment estimation B_t , MBAO-FAD undoubtedly introduces additional memory overhead compared with the optimization algorithms that do not require the second-order moment estimation, including MBGD-SGD, MBGD-AdaGrad, MBGD-RMSP, and MGBD-Momentum. Nevertheless, the additional memory overhead introduced by MBAO-FAD is acceptable, and this algorithm achieves substantial improvements in predictive performance, as presented in Table IV and Fig. 2.

F. Parameter Sensitivity Analysis

The parameter sensitivity experiments for the proposed MBAO-FAD are presented. The results are shown in Fig. 4.

Additionally, a separate comparison with the baseline algorithm MBGD-RDA is provided. According to Algorithm 2, MBAO-FAD has many hyperparameters such as λ , P , N_m , N_{bs} , and α . However, owing to space limitations, the parameter sensitivity experiments only consider key parameters that considerably affect performance metrics.

According to the aforementioned experimental results, the minibatch size N_{bs} substantially influences the performance of MBAO-FAD, as shown in Fig. 4(a) and Fig. 4(b). The proposed MBAO-FAD algorithm achieves stable RMSE metrics under different N_{bs} settings. Moreover, with increasing N_{bs} , the gap in computational time cost between MBAO-FAD and MBGD-RDA further increases. As illustrated in Fig. 4(c), MBAO-FAD maintains stable learning performance under different initial learning rates α . Moreover, Fig. 4(d) shows that although the performance gap between MBAO-

FAD and MBGD-RDA varies across different input dimensions, MBO-FAD outperforms MBGD-RDA in terms of all three key metrics.

V. CONCLUSIONS AND FUTURE WORK

TSK fuzzy systems have emerged as a prominent approach in machine learning for regression and classification tasks owing to their strong nonlinear approximation capabilities and inherent interpretability. However, they still face challenges in terms of computational inefficiency in parameter optimization and training. Therefore, this study proposed a novel MBO-FAD algorithm for designing efficient TSK fuzzy systems. First, forward automatic differentiation was applied to a TSK fuzzy system representing a completely different approach from traditional optimization techniques. Specifically, a fuzzy parameter gradient estimation method based on forward automatic differentiation was proposed, aiming to quickly calculate the loss value and its gradient in forward propagation using a matrix-free Jacobian-vector product. This indicates that MBO-FAD can completely mitigate the computational costs associated with backward propagation. Furthermore, the non-negative matrix factorization-based adaptive parameter update strategy was proposed for substantially reducing computational memory overhead.

This study demonstrates that MBO-FAD can achieve machine learning tasks of TSK fuzzy system without the need for backward propagation. We assert that MBO-FAD not only requires less computational time but also exhibits computational competitiveness, especially in comparison with the current state-of-the-art algorithm MBGD-RDA. Comparative experiments across seven methods on the commonly applied public nine datasets indicate that the proposed MBO-FAD achieves overall optimal performance in terms of optimization accuracy (RMSE testing loss) and efficiency (computational time cost and computational memory overhead).

Notably, TSK fuzzy systems have a substantial advantage in interpretability compared to mainstream neural network models. However, this study did not delve into whether MBO-FAD can further enhance its interpretability. Therefore, future studies must investigate whether our proposed MBO-FAD combined with rule-based fuzzy logic contributes to understanding the biological learning mechanisms in the brain as backpropagation-based optimization algorithms have traditionally been deemed implausible.

REFERENCES

- [1] E. Mamdani and S. Assilian, "An experiment in linguistic synthesis with a fuzzy logic controller," *Int. J. Man Mach. Stud.*, vol. 7, no. 1, pp. 1–13, 1975.
- [2] T. Takagi and M. Sugeno, "Fuzzy identification of systems and its applications to modeling and control," *IEEE Trans. Syst. Man Cybern.*, vol. SMC-15, no. 1, pp. 116–132, 1985.
- [3] Y. Zhang, G. Wang, T. Zhou, X. Huang, S. Lam, J. Sheng, K. S. Choi, J. Cai, and W. Ding, "Takagi-sugeno-kang fuzzy system fusion: A survey at hierarchical, wide and stacked levels," *Inf. Fusion*, vol. 101, p. 101977, 2024.
- [4] J. Xie, Q. Deng, S. Xia, Y. Zhao, G. Wang, and X. Gao, "Research on efficient fuzzy clustering method based on local fuzzy granular balls," *ArXiv*, vol. abs/2303.03590, 2023.
- [5] L. Jiao, H. Yang, Z. ga Liu, and Q. Pan, "Interpretable fuzzy clustering using unsupervised fuzzy decision trees," *Inf. Sci.*, vol. 611, pp. 540–563, 2022.
- [6] X. Han, X. Zhu, W. Pedrycz, and Z. Li, "A three-way classification with fuzzy decision trees," *Appl. Soft Comput.*, vol. 132, p. 109788, 2023.
- [7] C. Fernandez-Basso, M. D. Ruiz, and M. J. Martin-Bautista, "Spark solutions for discovering fuzzy association rules in big data," *Int. J. Approximate Reasoning*, vol. 137, pp. 94–112, 2021.
- [8] M. D. Vasilakakis and D. K. Iakovidis, "Fuzzy similarity phrases for interpretable data classification," *Inf. Sci.*, vol. 624, pp. 881–907, 2023.
- [9] H. Zhao, Y. Wu, and W. Deng, "An interpretable dynamic inference system based on fuzzy broad learning," *IEEE Trans. Instrum. Meas.*, vol. 72, pp. 1–12, 2023.
- [10] H. Xia, J. Tang, W. Yu, C. Cui, and J. Qiao, "Takagi-sugeno fuzzy regression trees with application to complex industrial modeling," *IEEE Trans. Fuzzy Syst.*, vol. 31, no. 7, pp. 2210–2224, 2023.
- [11] T. Deneux, "Quantifying prediction uncertainty in regression using random fuzzy sets: The ennreg model," *IEEE Trans. Fuzzy Syst.*, vol. 31, no. 10, pp. 3690–3699, 2023.
- [12] H. Bayarassou and A. Megri, "New approach based on a fuzzy regression model for a photovoltaic system," *Electr. Power Syst. Res.*, vol. 217, p. 109091, 2023.
- [13] J.-S. Jang, "Anfis: adaptive-network-based fuzzy inference system," *IEEE Trans. Syst. Man Cybern.*, vol. 23, no. 3, pp. 665–685, 1993.
- [14] C. Li, J. Gao, J. Yi, and G. Zhang, "Analysis and design of functionally weighted single-input-rule-modules connected fuzzy inference systems," *IEEE Trans. Fuzzy Syst.*, vol. 26, no. 1, pp. 56–71, 2018.
- [15] R. Khemchandani, Jayadeva, and S. Chandra, "Regularized least squares fuzzy support vector regression for financial time series forecasting," *Expert Syst. Appl.*, vol. 36, no. 1, pp. 132–138, 2009.
- [16] S.-B. Roh, S.-K. Oh, W. Pedrycz, and Z. Fu, "Design of fuzzy ensemble architecture realized with the aid of fcm-based fuzzy partition and nn with weighted lse estimation," *IEEE Trans. Fuzzy Syst.*, vol. 29, no. 3, pp. 569–583, 2021.
- [17] D. Wu and W. Wan Tan, "Genetic learning and performance evaluation of interval type-2 fuzzy logic controllers," *Eng. Appl. Artif. Intell.*, vol. 19, no. 8, pp. 829–841, 2006.
- [18] T. Zhao, C. Chen, H. Cao, S. Dian, and X. Xie, "Multiobjective optimization design of interpretable evolutionary fuzzy systems with type self-organizing learning of fuzzy sets," *IEEE Trans. Fuzzy Syst.*, vol. 31, no. 5, pp. 1638–1652, 2023.
- [19] V. K. Maurya and S. J. Nanda, "Time-varying multi-objective smart home appliances scheduling using fuzzy adaptive dynamic spea2 algorithm," *Eng. Appl. Artif. Intell.*, vol. 121, p. 105944, 2023.
- [20] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, pp. 533–536, 1986.
- [21] L.-X. Wang and J. Mendel, "Fuzzy basis functions, universal approximation, and orthogonal least-squares learning," *IEEE Trans. Neural Networks*, vol. 3, no. 5, pp. 807–814, 1992.
- [22] I. Eyoh, R. John, G. De Maere, and E. Kayacan, "Hybrid learning for interval type-2 intuitionistic fuzzy logic systems as applied to identification and prediction problems," *IEEE Trans. Fuzzy Syst.*, vol. 26, no. 5, pp. 2672–2685, 2018.
- [23] Y. Li, R. Gault, and T. M. McGinnity, "Probabilistic, recurrent, fuzzy neural network for processing noisy time-series data," *IEEE Trans. Neural Networks Learn. Syst.*, vol. 33, no. 9, pp. 4851–4860, 2022.
- [24] D. Wu, Y. Yuan, J. Huang, and Y. Tan, "Optimize tsk fuzzy systems for regression problems: Minibatch gradient descent with regularization, droprule, and adabound (mbgd-rda)," *IEEE Trans. Fuzzy Syst.*, vol. 28, no. 5, pp. 1003–1015, 2020.
- [25] Y. Cui, D. Wu, and J. Huang, "Optimize tsk fuzzy systems for classification problems: Minibatch gradient descent with uniform regularization and batch normalization," *IEEE Trans. Fuzzy Syst.*, vol. 28, no. 12, pp. 3065–3075, 2020.
- [26] S. Jawed, I. Faye, and A. S. Malik, "Deep learning-based assessment model for real-time identification of visual learners using raw eeg," *IEEE Trans. Neural Syst. Rehabil. Eng.*, vol. 32, pp. 378–390, 2024.
- [27] X. Peng, L. Li, and F.-Y. Wang, "Accelerating minibatch stochastic gradient descent using typicality sampling," *IEEE Trans. Neural Networks Learn. Syst.*, vol. 31, no. 11, pp. 4649–4659, 2020.
- [28] X. Zhou, L. Liang, J. Zhang, C.-K. Wen, and S. Jin, "Mini-batch gradient-based mcmc for decentralized massive mimo detection," *IEEE Trans. Commun.*, vol. 73, no. 1, pp. 677–692, 2025.
- [29] J. Duchi, E. Hazan, and Y. Singer, "Adaptive subgradient methods for online learning and stochastic optimization," *J. Mach. Learn. Res.*, vol. 12, no. null, p. 2121–2159, Jul. 2011.

- [30] L. Luo, Y. Xiong, Y. Liu, and X. Sun, "Adaptive gradient methods with dynamic bound of learning rate," in *Proc. Int. Conf. Learn. Represent.*, New Orleans, LA, USA, May 2019, presented at.
- [31] Z. Shi, S. Huang, L. Wu, Q. Zhang, X. Zhang, Y. Cao, Y. Chen, and Y. Lv, "Tsk fuzzy system optimization for high-dimensional regression problems," *IEEE Trans. Emerging Top. Comput. Intell.*, vol. 9, no. 2, pp. 1569–1580, 2025.
- [32] Y. Cui, Y. Xu, R. Peng, and D. Wu, "Layer normalization for tsf fuzzy system optimization in regression problems," *IEEE Trans. Fuzzy Syst.*, vol. 31, no. 1, pp. 254–264, 2023.